

Operators, Conditions and Decision Making in Python

Contents

1	Introduction	3
2	Arithmetic Operators	3
2.1	Basic Setup	3
2.2	Addition	3
2.3	Subtraction	3
2.4	Multiplication	4
2.5	Division	4
2.6	Integer Division	4
2.7	Modulo Operator	4
2.8	Exponent Operator	5
2.9	Mixed Expression	5
3	Comparison Operators	5
3.1	Greater Than	5
3.2	Less Than	6
3.3	Greater Than or Equal To	6
3.4	Less Than or Equal To	6
3.5	Equal To	6
3.6	Not Equal To	6
4	Chained Comparisons	6
5	Logical Operators	7
5.1	AND Operator	7
5.2	OR Operator	8
5.3	NOT Operator	8
6	Real Example: Pass or Fail System	8
7	Indentation	9
8	If Else Statements	9
8.1	Example	9
8.2	Colon Symbol	9

9	Rules for Naming Variables	10
9.1	Rule 1: Start with Letter or Underscore	10
9.2	Rule 2: Variable Name Must be Single Word	10
9.2.1	Snake Case	10
9.2.2	Camel Case	10
9.2.3	Pascal Case	10
9.3	Rule 3: Cannot Use Reserved Keywords	10
10	Ternary Operator	11
11	Example 2: Grade Calculator	11
12	Understanding Elif	12
13	Why Validation Matters	12
14	Final Thoughts	13
15	Exercises	14
15.1	Exercise 1: Simple Calculator	14
15.2	Exercise 2: Even or Odd	14
15.3	Exercise 3: Voting Eligibility	14
15.4	Exercise 4: Positive, Negative or Zero	14
15.5	Exercise 5: Largest Number	14
15.6	Exercise 6: Pass or Fail	15
15.7	Exercise 7: Temperature Checker	15
15.8	Exercise 8: Mini Login System	15
15.9	Exercise 9: Simple Grade System	15
15.10	Exercise 10: Challenge Exercise	15
16	Homework Suggestion	16

1 Introduction

Till now, we learned:

- Data types
- Variables
- Input and Output

But programming becomes truly interesting when programs start making decisions.
Questions like:

- Did student pass?
- Is password correct?
- Is age greater than 18?

For all these things, we need:

- Operators
- Conditions
- Decision making

So today, we enter the world of logic.

2 Arithmetic Operators

Arithmetic operators are used for mathematical calculations.

2.1 Basic Setup

```
a = 5  
b = 10
```

2.2 Addition

```
print(a + b)
```

Output:

```
15
```

2.3 Subtraction

```
print(a - b)
```

Output:

```
-5
```

2.4 Multiplication

```
print(a * b)
```

Output:

```
50
```

2.5 Division

```
print(a / b)
```

Output:

```
0.5
```

Normal division always gives floating point result.

2.6 Integer Division

```
print(a // b)
```

Output:

```
0
```

This gives only quotient.

No decimal part.

That is why it is called integer division.

2.7 Modulo Operator

```
print(a % b)
```

Output:

```
5
```

Modulo gives remainder.

Example:

```
10 % 3
```

Result:

```
1
```

Why is modulo useful?

- Checking even or odd numbers
- Cyclic operations
- Timer calculations

2.8 Exponent Operator

```
print(a ** b)
```

This means:

$$a^b$$

Example:

```
2 ** 3
```

Result:

```
8
```

Because:

$$2 \times 2 \times 2 = 8$$

2.9 Mixed Expression

```
print(a + b/2 + 4 - 3)
```

Python follows mathematical precedence rules.
Just like mathematics:

- Division first
- Then addition/subtraction

3 Comparison Operators

Comparison operators compare values.
Their result is always:

- True
- False

Meaning:

- Boolean values

3.1 Greater Than

```
print(5 > 2)
```

Output:

```
True
```

3.2 Less Than

```
print(5 < 2)
```

Output:

```
False
```

3.3 Greater Than or Equal To

```
print(5 >= 2)
```

3.4 Less Than or Equal To

```
print(5 <= 2)
```

3.5 Equal To

```
print(5 == 2)
```

Important:

- = means assignment
- == means comparison

Many beginners accidentally write:

```
if marks = 50
```

And Python becomes angry immediately.

3.6 Not Equal To

```
print(5 != 2)
```

Meaning:

“5 is not equal to 2”

4 Chained Comparisons

Python allows beautiful comparisons.

```
print(2 < 5 < 10)
```

This means:

2 is less than 5 AND 5 is less than 10

Output:

```
True
```

Another example:

```
print(2 < 5 < 3)
```

Output:

```
False
```

Because:

$5 < 3$

is false.

5 Logical Operators

Sometimes one condition is not enough.

We combine conditions using:

- and
- or
- not

5.1 AND Operator

All conditions must be True.

```
print(5 > 2 and 5 > 3 and 5 < 10)
```

Output:

```
True
```

Because:

- $5 > 2 = \text{True}$
- $5 > 3 = \text{True}$
- $5 < 10 = \text{True}$

All are True.

Now this:

```
print(5 > 2 and 5 < 3 and 5 > 10)
```

Output:

```
False
```

Even one False makes whole expression False.

AND operator is very strict.

Like that one teacher who cuts marks for one spelling mistake.

5.2 OR Operator

At least one condition must be True.

```
print(5 > 2 or 5 > 3)
```

Output:

```
True
```

```
print(5 < 2 or 5 < 3)
```

Output:

```
False
```

5.3 NOT Operator

Reverses result.

```
print(not 5 > 2)
```

- $5 > 2 = \text{True}$
- $\text{not True} = \text{False}$

Output:

```
False
```

6 Real Example: Pass or Fail System

Now let's make program decisions.

```
english_marks = int(input("Enter marks obtained in English: "))
english_pass_marks = 32

english_passed = english_marks >= english_pass_marks
```

Suppose user enters:

```
34
```

Then:

$$34 \geq 32$$

Result:

```
True
```

So:

```
english_passed
```

stores:

```
True
```

7 Indentation

Now comes something extremely important in Python.

```
if english_passed:
    print("This student has passed")
```

Notice the space before `print()`.
That spacing is called:

Indentation

Python uses indentation to understand blocks of code.
Unlike some languages using many brackets:

```
{
}
```

Python says:

“Use proper spacing like civilized people.”

8 If Else Statements

Programs can make decisions using:

- if
- else

8.1 Example

```
if english_passed:
    print("This student has passed english subject")
    print("Congrats")
else:
    print("This student has failed english subject")
    print("Sorry, please try again next time")
```

8.2 Colon Symbol

Notice this:

```
if english_passed:
```

The `:` symbol is called colon.
It tells Python:

“A block of code starts now.”

9 Rules for Naming Variables

Choosing variable names properly is important.

Future-you will thank present-you.

9.1 Rule 1: Start with Letter or Underscore

Correct:

```
rollno1
_rollno
```

Wrong:

```
1rollno
```

9.2 Rule 2: Variable Name Must be Single Word

Python does not allow spaces.

Wrong:

```
english marks
```

Instead we use naming styles.

9.2.1 Snake Case

```
english_marks
```

Most common in Python.

9.2.2 Camel Case

```
englishMarks
```

9.2.3 Pascal Case

```
EnglishMarks
```

9.3 Rule 3: Cannot Use Reserved Keywords

Wrong examples:

```
if = 5
and = 4
or = 3
not = 2
```

These words already have special meaning in Python.

10 Ternary Operator

Sometimes writing full if-else feels too big.

Python gives short form.

```
print("Pass" if english_marks > 32 else "Fail")
```

Meaning:

- If condition True → print Pass
- Else → print Fail

Small, compact, powerful.

Like instant noodles of programming.

11 Example 2: Grade Calculator

Now let us build a bigger program.

```
english_marks = int(input("Enter marks obtained in English: "))

if english_marks > 100:
    print("Marks greater than 100 not allowed")
elif english_marks > 90:
    print("A+")
elif english_marks > 80:
    print("A")
elif english_marks > 70:
    print("B+")
elif english_marks > 60:
    print("B")
elif english_marks > 50:
    print("C+")
elif english_marks > 40:
    print("C")
elif english_marks > 30:
    print("D")
elif english_marks > 20:
    print("E")
elif english_marks > 0:
    print("NG")
else:
    print("Negative marks not allowed")
```

12 Understanding Elif

`elif` means:

Else If

Python checks conditions from top to bottom.
As soon as one condition becomes True:

- Python executes that block
- Remaining conditions are skipped

Example:

Input:

```
85
```

Python checks:

- Is $85 > 100$? No
- Is $85 > 90$? No
- Is $85 > 80$? Yes

So output:

```
A
```

And remaining conditions are skipped.

13 Why Validation Matters

Notice this condition:

```
if english_marks > 100:
```

This is called validation.
Programs should not blindly trust user input.
Users can enter:

- 999
- -500
- Strange values

Validation protects program correctness.

14 Final Thoughts

Today you learned:

- Arithmetic operators
- Comparison operators
- Logical operators
- Conditions
- if-else statements
- Indentation
- Variable naming rules
- Ternary operator
- Grade calculator logic

This is a huge milestone.

Because now your programs can:

Think and decide.

Well... not exactly think.

But at least pretend convincingly.

15 Exercises

15.1 Exercise 1: Simple Calculator

Take two numbers from user and print:

- Addition
- Subtraction
- Multiplication
- Division

15.2 Exercise 2: Even or Odd

Take number input from user.

Use modulo operator to check:

- Even
- Odd

Hint:

```
number % 2
```

15.3 Exercise 3: Voting Eligibility

Take user's age as input.

If age is greater than or equal to 18:

- Print "Eligible to vote"

Else:

- Print "Not eligible"

15.4 Exercise 4: Positive, Negative or Zero

Take number input and check:

- Positive
- Negative
- Zero

15.5 Exercise 5: Largest Number

Take two numbers from user.

Print which one is greater.

15.6 Exercise 6: Pass or Fail

Take marks input.

If marks greater than or equal to 32:

- Print Pass

Else:

- Print Fail

15.7 Exercise 7: Temperature Checker

Take temperature input.

- Above 35 → “Hot day”
- Between 20 and 35 → “Normal day”
- Below 20 → “Cold day”

15.8 Exercise 8: Mini Login System

Store:

```
correct_password = "python123"
```

Take password input from user.

If password matches:

- Print “Login Successful”

Else:

- Print “Wrong Password”

15.9 Exercise 9: Simple Grade System

Take marks input and print:

- A for marks above 80
- B for marks above 60
- C for marks above 40
- Fail otherwise

15.10 Exercise 10: Challenge Exercise

Take three numbers from user.

Print the largest number among them.

(Hint: use logical operators)

16 Homework Suggestion

Try changing values in all examples.

Programming is learned by:

Typing, running, breaking things, fixing things.

Not by only reading notes like a history book.